Linguaggi di Programmazione

Espressioni regolari, NFA, DFA e grammatiche regolari rappresentano tutti e soli i linguaggi regolari

Roberto Gorrieri

Alma Mater Studiorum - Università di Bologna

Da espressioni regolari a NFA equivalenti

Da espressioni regolari a NFA equivalenti

Teorema
Data una espressione regolare s, possiamo costruire un NFA $N[s]$ tale che $\mathcal{L}[s] = L[N[s]]$.

Ovvero, gli NFA riconoscono "tutti" i linguaggi regolari! (Vedremo che gli NFA riconoscono "solo" i linguaggi regolari.)

Dimostrazione: Per induzione sulla sintassi (astratta) della espressione regolare s, costruiremo un NFA $N[s]$ equivalente a s, in modo da mantenere i seguenti due invarianti (per semplificare la costruzione e garantirne la correttezza):

- lo stato iniziale di $N[s]$ non ha archi entranti;
- $N[s]$ ha un solo stato finale senza archi uscenti.

Dimostrazione: casi semplici

$s = \emptyset$ $N[s]$



Osserva che $\mathcal{L}[\emptyset] = \emptyset = L[N[\emptyset]]$

$s = \epsilon$ $N[s]$



Osserva che $\mathcal{L}[\epsilon] = \{\epsilon\} = L[N[\epsilon]]$

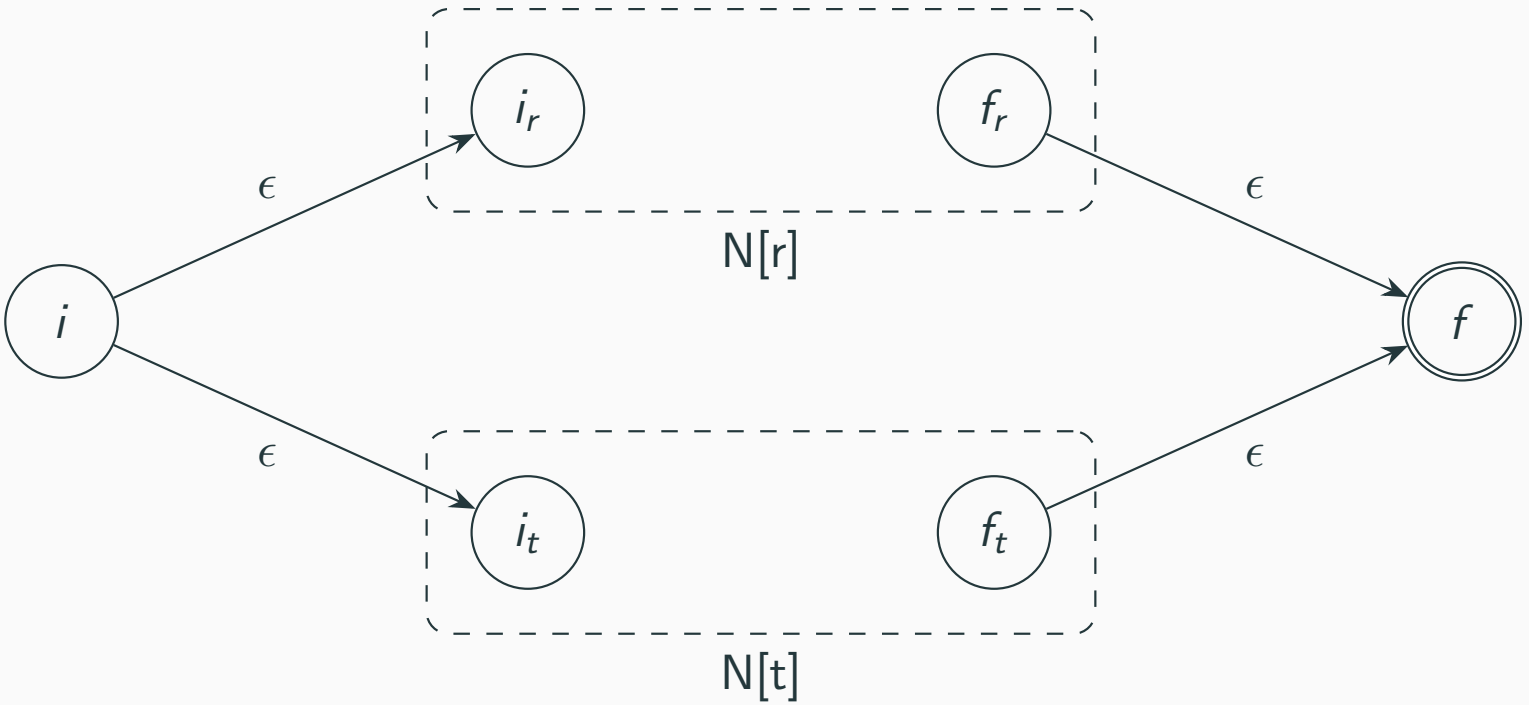
$s = a$ $N[s]$



Osserva che $\mathcal{L}[a] = \{a\} = L[N[a]]$

Dimostrazione: caso $s = r|t$

$s = r \mid t$ $\quad$ $N[s]$

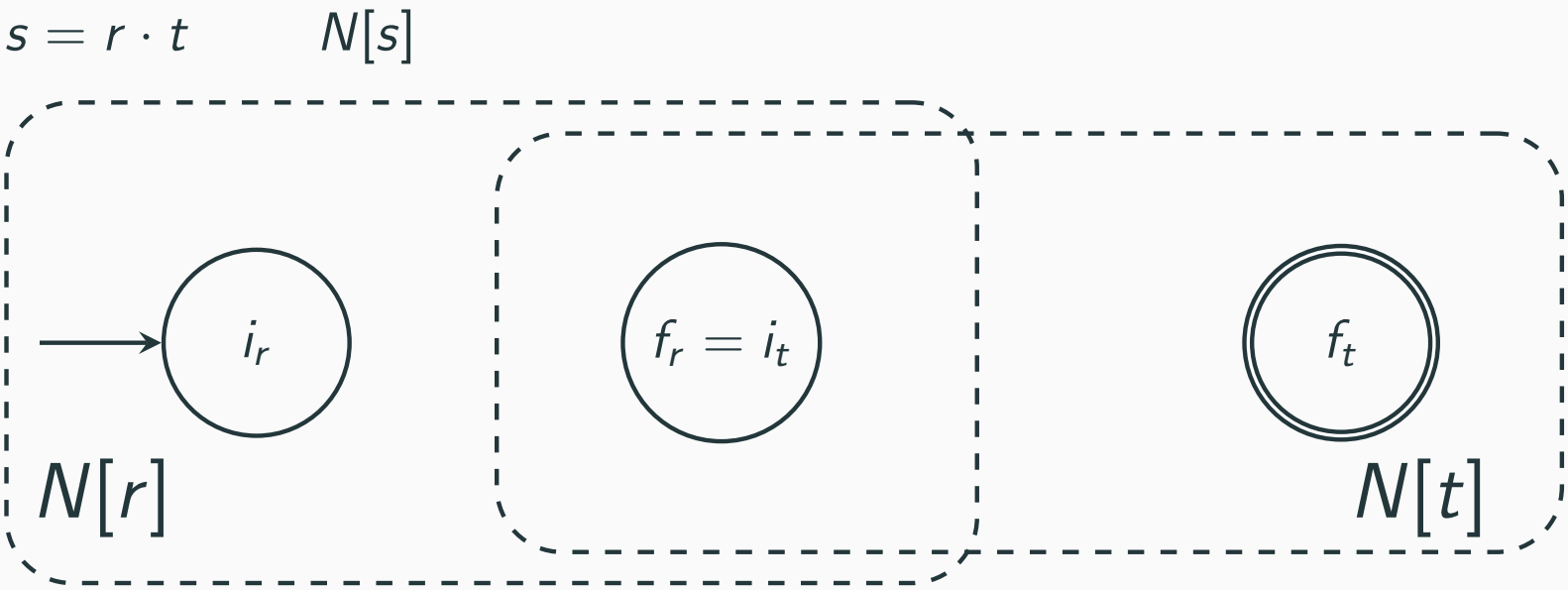


Osserva che

$$\mathcal{L}[r|t] = \mathcal{L}[r] \cup \mathcal{L}[t] = L[N[r]] \cup L[N[t]] = L[N[r|t]]$$

per ipotesi induttiva

Dimostrazione: caso $s = r \cdot t$



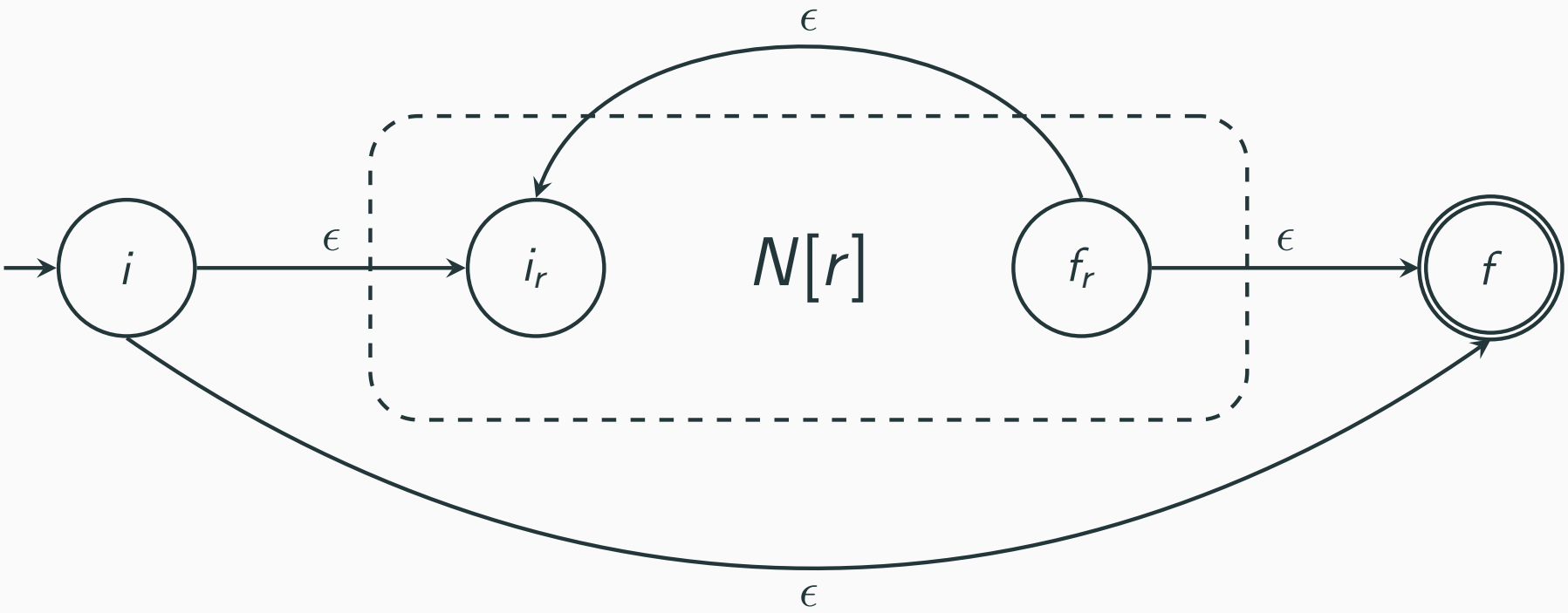
Osserva che abbiamo fuso il finale di $N[r]$ con l'iniziale di $N[t]$. La costruzione è corretta grazie ai due invarianti.

$$\mathcal{L}[r \cdot t] = \mathcal{L}[r] \cdot \mathcal{L}[t] = L[N[r]] \cdot L[N[t]] = L[N[r \cdot t]]$$

per ipotesi induttiva

Dimostrazione: caso $s = r^*$

$s = r^*$ $N[s]$



Osserva che

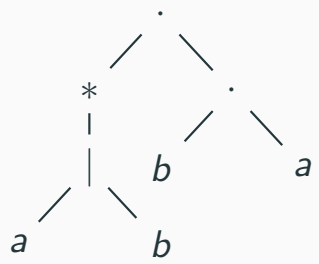
$$\mathcal{L}[r^*] = (\mathcal{L}[r])^* = (L[N[r]])^* = L[N[r^*]]$$

per ipotesi induttiva

E questo caso conclude la prova: c.v.d.

Esempio

Consideriamo $s = (a|b)^*ba$, il cui albero sintattico è



Ricostruiamo bottom-up il corrispondente NFA.

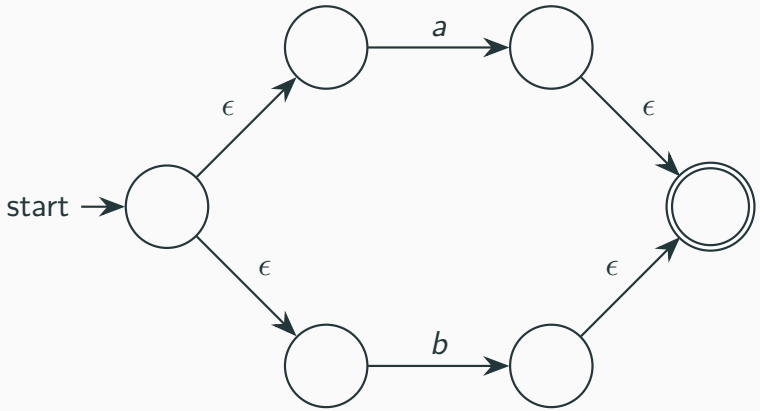
$N[a]$



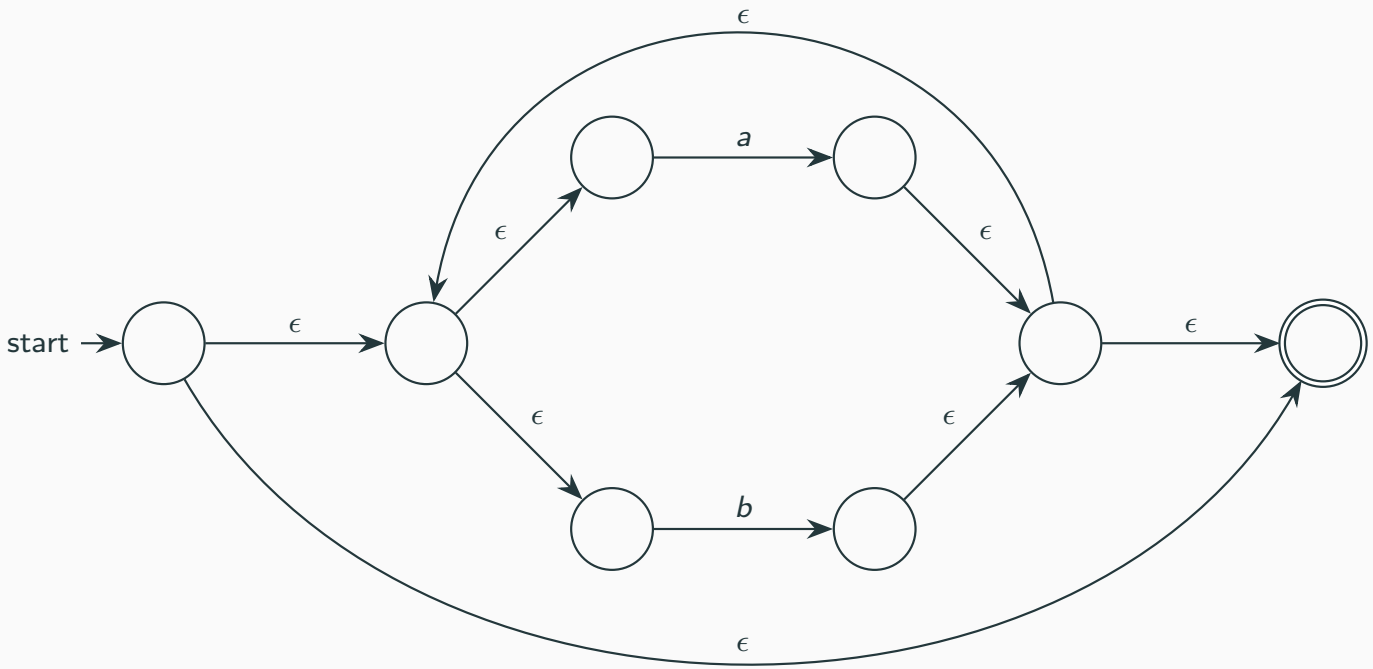
$N[b]$



$N[a|b]$



Esempio – continua



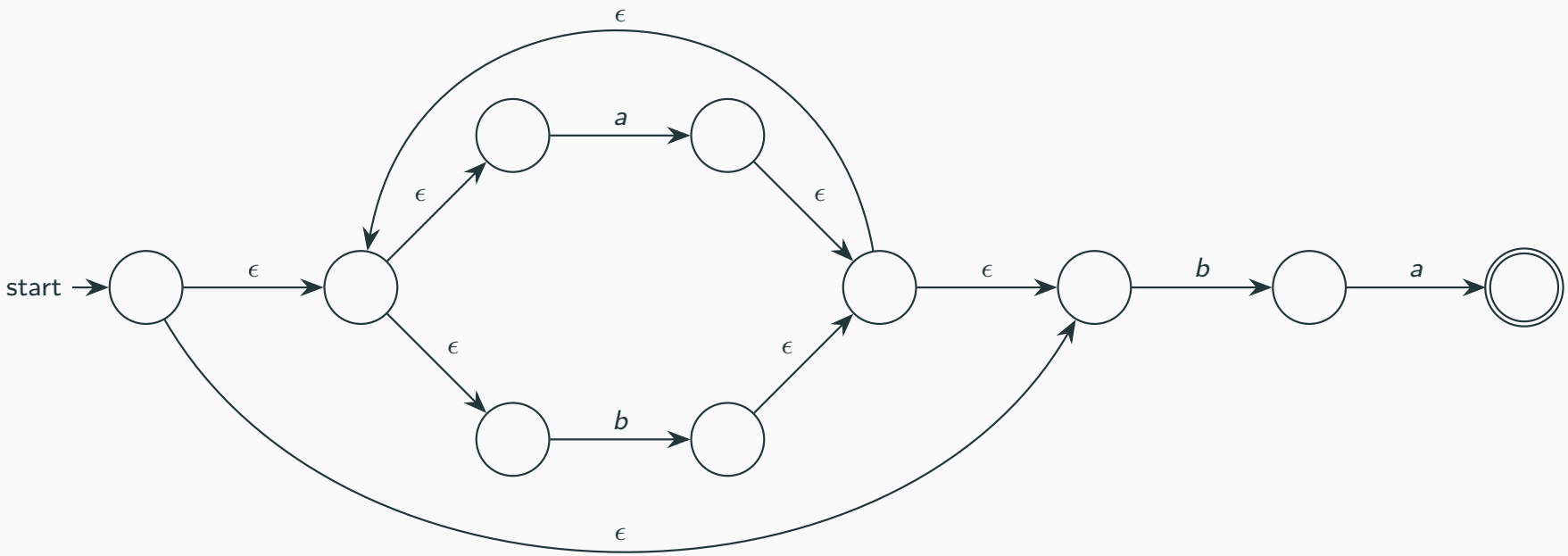
$N[(a|b)^*]$

$N[ba]$



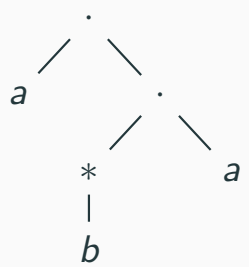
Esempio – continua (2)

$N[(a|b)^* ba]$

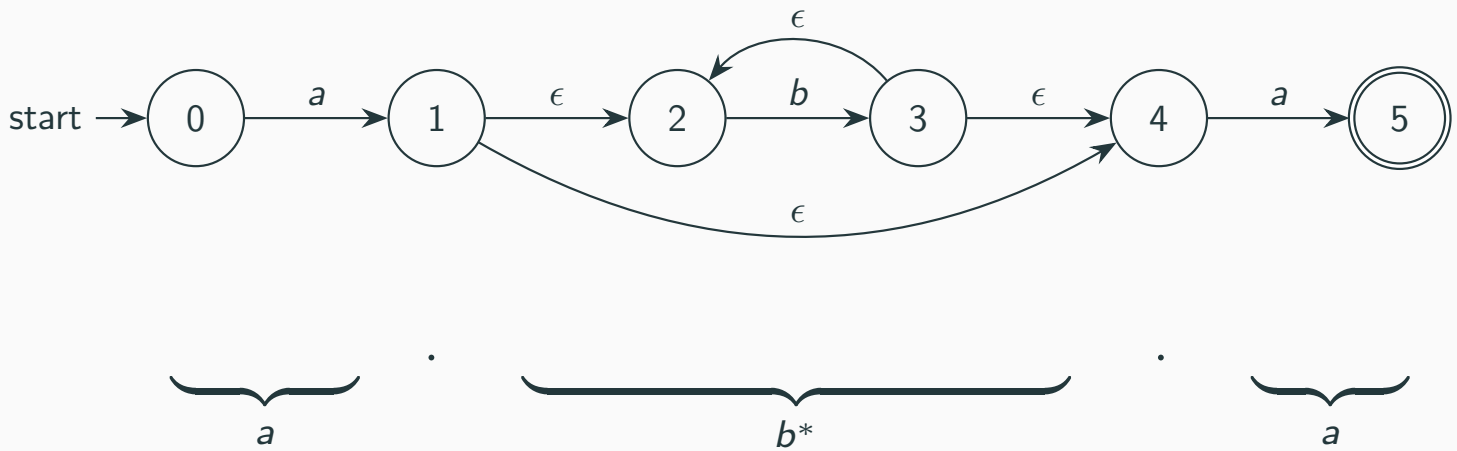


Altro Esempio

Consideriamo $s = ab^*a$, il cui albero sintattico è



e il suo NFA $N[s]$ è il seguente:



Altro Esempio – continua

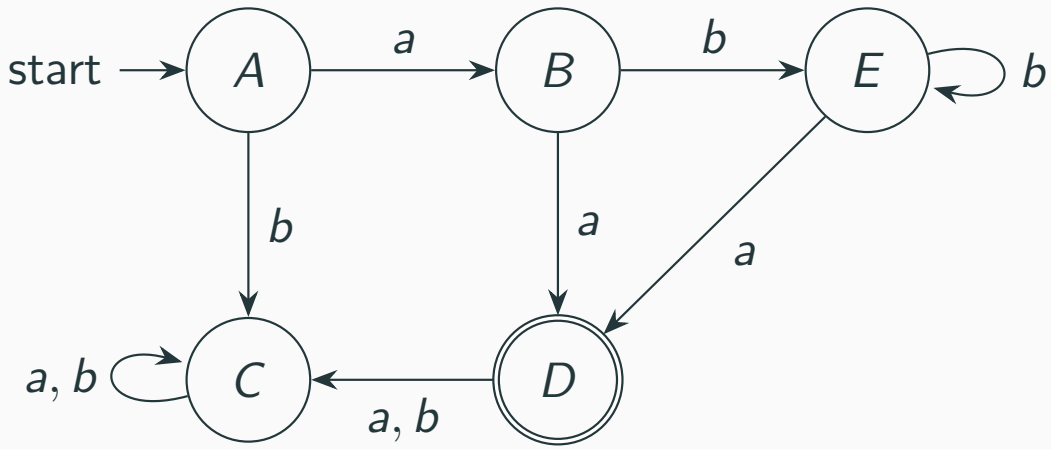
Costruiamo il DFA M_N associato a $N[ab^*a]$.

- Lo stato iniziale del DFA è dato da 0, più tutti gli stati che posso raggiungere da 0 con mosse ϵ:

  $A = \epsilon\text{-}closure(\{0\}) = \{0\}$.
- Per calcolare lo stato che raggiungo da A leggendo a, devo vedere quali stati in A possono fare a, e poi fare la ϵ-closure di quelli raggiunti con quelle mosse etichettate a.

  $\Delta(A, a) = \epsilon\text{-}closure(\{1\}) = \{1, 2, 4\} = B$
- $\Delta(A, b) = \epsilon\text{-}closure(\emptyset) = \emptyset = C$
- $\Delta(B, a) = \epsilon\text{-}closure(\{5\}) = \{5\} = D$
- $\Delta(B, b) = \epsilon\text{-}closure(\{3\}) = \{3, 4, 2\} = E$
- $\Delta(C, a) = \epsilon\text{-}closure(\emptyset) = \emptyset = C$
- $\Delta(C, b) = \epsilon\text{-}closure(\emptyset) = \emptyset = C$
- $\Delta(D, a) = \epsilon\text{-}closure(\emptyset) = \emptyset = C$
- $\Delta(D, b) = \epsilon\text{-}closure(\emptyset) = \emptyset = C$
- $\Delta(E, a) = \epsilon\text{-}closure(\{5\}) = \{5\} = D$
- $\Delta(E, b) = \epsilon\text{-}closure(\{3\}) = \{3, 4, 2\} = E$

Altro Esempio – continua (2)



DFA M_N

Nota che $D = \{5\}$ è finale per M_N perché 5 è finale per N.
Nota anche che $C = \emptyset$ è lo stato pozzo (di errore).

Grammatiche regolari e relazioni con NFA e DFA

Grammatiche regolari

Grammatica regolare
Una grammatica libera $G = (NT, T, R, S)$ è **regolare** se e solo se ogni produzione in R è della forma $A \to aB$ oppure $A \to a$, oppure $A \to \epsilon$, dove $A, B \in NT$ ed $a \in T$.

Notare che ogni forma sentenziale α derivabile dal simbolo iniziale S (cioè tale che $S \Rightarrow^* \alpha$) è tale che una di queste sia sicuramente soddisfatta:

- $\alpha = wB$, con $w \in T^+$ e $B \in NT$, oppure
- $\alpha = w$, con $w \in T^+$, oppure
- $\alpha = \epsilon$.

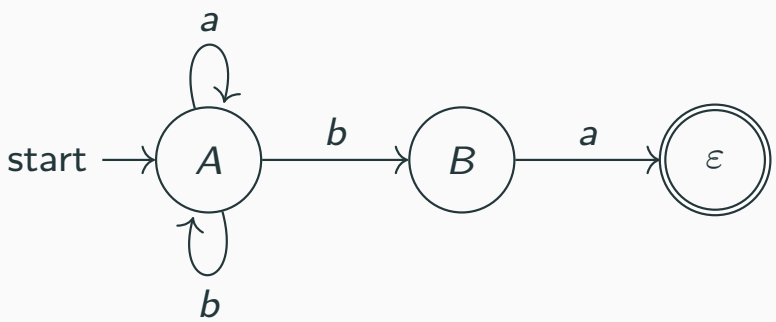
Quindi, se $S \Rightarrow^* \alpha$, allora in α può occorrere al più un nonterminale che, se presente, si trova in ultima posizione (più a destra).

Esempio

$$A \to aA \mid bA \mid bB$$

$$B \to a$$

La grammatica G è regolare e $L(G) = \mathcal{L}[(a|b)^* ba]$, perché con le produzioni $A \to aA \mid bA$ posso generare una qualunque sequenza di a e b (lasciando A in ultima posizione a destra), che poi verrà conclusa dalle produzioni $A \to bB$ e $B \to a$.
Proviamo ad associare un NFA a G:



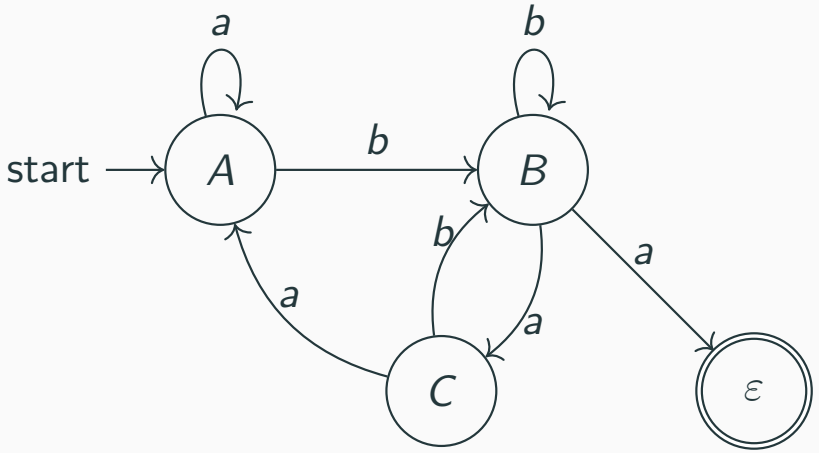
- abbiamo associato ad ogni nonterminale uno stato dell'NFA, e
- abbiamo aggiunto uno stato finale ε (da non confondere con la stringa vuota ϵ).

È possibile, in generale, associare ad ogni grammatica regolare un NFA equivalente?

Esempio (2)

$$A \to aA \mid bB$$
$$B \to bB \mid aC \mid a$$
$$C \to aA \mid bB$$

Questa grammatica G è regolare, e il suo NFA N_G equivalente associato è



Non è immediato riconoscere che anche per questo NFA il suo linguaggio riconosciuto è ancora $L[N_G] = \mathcal{L}[(a|b)^* ba]$.

Da grammatiche regolari a NFA equivalenti

Teorema
Data una grammatica regolare G, si può costruire un NFA N_G equivalente, cioè tale che $L(G) = L[N_G]$.

Dimostrazione: sia $G = (NT, T, R, S)$, allora $N_G = (\Sigma, Q, \delta, q_0, F)$ è definito come segue:

- $\Sigma = T$
- $Q = NT \cup \{\varepsilon\}$
- $F = \{\varepsilon\}$
- $q_0 = S$
- δ è definita come
  - $Z \in \delta(V, a)$ se $V \to aZ \in R$
  - $\varepsilon \in \delta(V, a)$ se $V \to a \in R$
  - $\varepsilon \in \delta(V, \epsilon)$ se $V \to \epsilon \in R$

Si può dimostrare (per induzione sulla lunghezza di $w \in T^*$) che con la grammatica G si può ottenere la derivazione $S \Rightarrow_G^* w$ se e solo se nell'automa N_G si ha la computazione $(S, w) \vdash_{N_G}^* (\varepsilon, \epsilon)$. Quindi, G e N_G sono equivalenti.

Da DFA a grammatiche regolari equivalenti

Teorema
Dato un DFA M possiamo definire una grammatica regolare G_M tale che $L[M] = L(G_M)$.

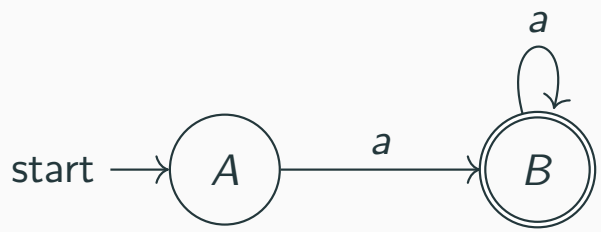
Dimostrazione: Dato il DFA $M = (\Sigma, Q, \delta, q_0, F)$, la grammatica associata $G_M = (NT, T, R, S)$ ha

- $NT = Q$
- $T = \Sigma$
- $S = q_0$
- per produzioni in R si procede come segue:
  - se $\delta(q_i, a) = q_j$, allora la produzione $q_i \to aq_j \in R$
  - se $q_i \in F$, allora anche $q_i \to \epsilon \in R$.

Si può dimostrare che $w \in L[M] \iff w \in L(G_M)$, cioè che nell'automa M si ha la computazione $(q_0, w) \vdash_M^* (q, \epsilon)$, con $q \in F$, se e solo se per la grammatica G_M si ha la derivazione $S \Rightarrow_{G_M}^* w$.

Esempio

Dato un DFA M il cui linguaggio $L[M]$ è descritto dall'espressione regolare aa^*:

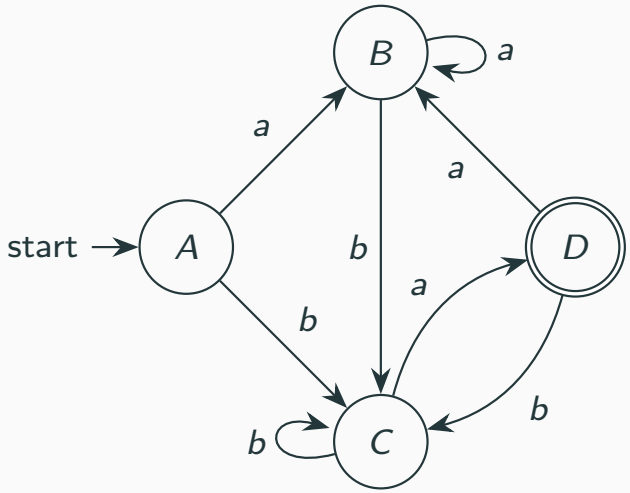


Secondo la costruzione del teorema appena visto, G_M è

$$A \to aB$$
$$B \to aB \mid \epsilon$$

Esempio (2)

Dato un DFA M' il cui linguaggio $L[M']$ è descritto dall'espressione regolare $(a|b)^*ba$:



Secondo la costruzione del teorema appena visto, $G_{M'}$ è

$$A \to aB \mid bC$$

$$B \to aB \mid bC$$

$$C \to bC \mid aD$$

$$D \to aB \mid bC \mid \epsilon$$

Domanda: è possibile costruire una G_M regolare partendo da un NFA (anziché da un DFA)? (Suggerimento: pensate alle transizioni ϵ.)

Da grammatiche regolari ad espressioni regolari equivalenti

Da grammatiche regolari ad espressioni regolari equivalenti

Teorema
Il linguaggio definito da una grammatica regolare G è un linguaggio regolare, cioè è possibile costruire una espressione regolare s_G tale che $L(G) = \mathcal{L}[s_G]$.

Dimostrazione (sketch): idea della prova

- caso semplice: un solo nonterminale, e.g., $A \to aA \mid b \mid \epsilon$
  È intuitivo vedere che $a^*(b|\epsilon)$ è la espressione regolare associata, perché con la produzione $A \to aA$ possiamo generare una sequenza arbitraria di a seguita da A, che può essere rimosso con le produzioni $A \to b \mid \epsilon$.
- caso medio: due nonterminali, e.g.,

$$A \to aA \mid bB \mid c$$
$$B \to cA \mid aB \mid d$$

  Si può vedere come un sistema da risolvere!
  Ricaviamo B dalla seconda "equazione": $B \approx a^*(cA|d)$ dove A compare nella espressione regolare associata

Prova: caso medio

Ora sostituiamo B con $a^*(cA|d)$ nella prima "equazione", ottenendo una "equazione" che usa solo A:

$$A \approx aA \mid ba^*(cA|d) \mid c$$

Con opportune manipolazioni su espressioni regolari, usando le leggi che abbiamo visto, possiamo scrivere

$$A \approx aA \mid ba^*cA \mid ba^*d \mid c$$

E quindi

$$A \approx (a|ba^*c)A \mid ba^*d \mid c$$

Ora siamo nella forma "semplice" e sappiamo come fare! L'espressione regolare associata ad A è

$$(a|ba^*c)^*(ba^*d|c)$$

Prova: sketch del caso generale

In generale, dati $NT = \{A_1, \ldots, A_n\}$ e $T = \{a_1, \ldots, a_k\}$, e la grammatica

$$A_1 \to a_{11}A_{11} \mid ... \mid a_{1p_1}A_{1p_1} \mid b_{11} \mid ... \mid b_{1q_1}\{+\epsilon\}$$
$$A_2 \to a_{21}A_{21} \mid ... \mid a_{2p_2}A_{2p_2} \mid b_{21} \mid ... \mid b_{2q_2}\{+\epsilon\}$$
$$\ldots$$
$$A_n \to a_{n1}A_{n1} \mid ... \mid a_{np_n}A_{np_n} \mid b_{n1} \mid ... \mid b_{nq_n}\{+\epsilon\}$$

dove tutti gli $a_{ij} \in T$, tutti i $b_{ij} \in T$, tutti gli $A_{ij} \in NT$, $0 \leq p_i \leq n \times k$, $0 \leq q_i \leq k$ e dove con $\{+\epsilon\}$ indichiamo la presenza opzionale della produzione epsilon, l'algoritmo parte con

$$A_n \approx s_n[A_1, ..., A_{n-1}]$$

cioè si costruisce una espressione regolare s_n per A_n che usa solo $A_1, ..., A_{n-1}$. Poi si procede sostituendo A_n con $s_n[A_1, ..., A_{n-1}]$ nell'equazione per A_{n-1}, in modo che poi si possa derivare

$$A_{n-1} \approx s_{n-1}[A_1, ..., A_{n-2}]$$

e così via fino ad arrivare ad A_1, che è il simbolo iniziale, la cui "equazione" si potrà risolvere perché saremo nel caso semplice.

Esempio

Consideriamo la grammatica regolare

$$A \to aB \mid \epsilon$$
$$B \to bA \mid \epsilon$$

Per B, l'espressione regolare associata è $bA \mid \epsilon$. Sostituisco questa al posto di B nella prima "equazione" e ottengo

$$A \approx a(bA \mid \epsilon) \mid \epsilon$$

Ora manipolo l'espressione regolare (usando leggi dell'equivalenza di espressioni regolari), ottenendo

$$A \approx abA \mid a \mid \epsilon$$

Siamo ora nel caso semplice ed otteniamo che l'espressione associata ad A è $(ab)^*(a \mid \epsilon)$.

Esempio (2)

Consideriamo la grammatica regolare

$$A \to aB \mid bA \mid d$$
$$B \to aB \mid c$$

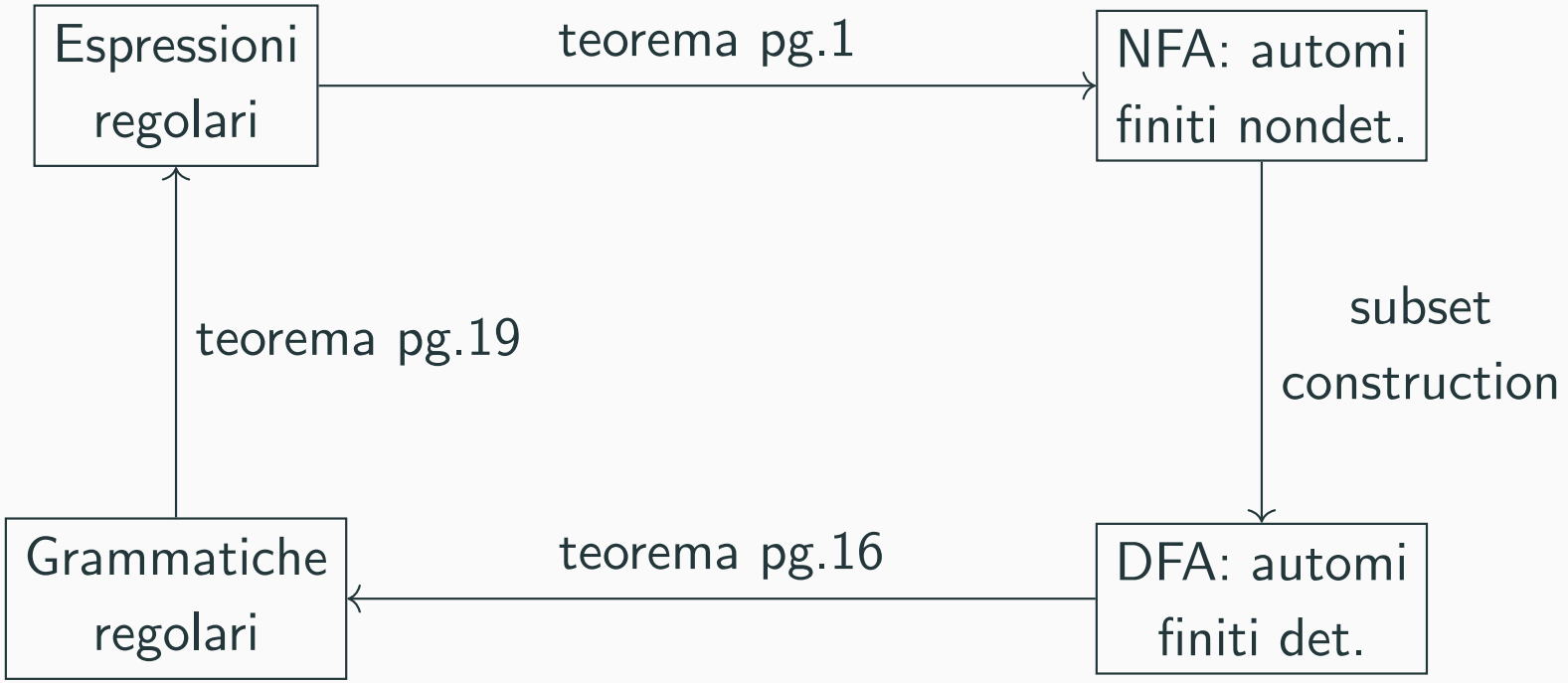
Per B, l'espressione regolare associata è a^*c (caso semplice). Sostituisco questa al posto di B nella prima "equazione" e ottengo

$$A \approx a(a^*c) \mid bA \mid d$$

Ora siamo ora nel caso semplice ed otteniamo che l'espressione associata ad A è $b^*(a(a^*c) \mid d)$.

N.B.: La grammatica con unica produzione $A \to aA$ definisce il linguaggio vuoto, non a^*, perché non c'è nessuna produzione che faccia sparire A. Secondo l'algoritmo, l'espressione regolare che si otterrebbe sarebbe $a^* \cdot \emptyset$, che è equivalente a $\emptyset$.

Riassumendo



Conseguenza: tutti questi formalismi sono equivalenti. Tutti generano (grammatiche regolari) / riconoscono (NFA/DFA) / descrivono (espressioni regolari) la stessa classe di linguaggi, ovvero i **linguaggi REGOLARI**.